1. Create the Redshift Cluster

1. **Login to AWS Console** → search for **Amazon Redshift**.
2. Click **Create cluster**.
3. Fill in details:
   - **Cluster identifier**: my-redshift-cluster
   - **Node type**: ra3.xlplus (RA3 is recommended for cost efficiency and storage scaling).
   - **Number of nodes**: 1 (single-node cluster).
   - **Database name**: dev (default, or choose your own).
   - **Master username**: e.g., adminuser
   - **Password**: Choose a secure password (note it down).
   - **Port**: 5439 (default for Redshift).
4. Choose **VPC** and subnet (default VPC is fine if just practicing).
5. Scroll down and click **Create cluster**.

---

2. Wait for Cluster to be Available

- Status will change from **Creating** → **Available**.
- This may take **2–5 minutes**.

---

3. Modify Security Group (Fix Connection Issue)

Most people face connection problems because the cluster blocks outside traffic. To fix this:

1. Go to your Redshift cluster → **Properties** tab.
2. Find the **VPC security groups** → click the linked security group.
3. In **EC2 → Security Groups → Inbound rules**:
   - Click **Edit inbound rules**.
   - Add rule:
     - **Type**: PostgreSQL
     - **Protocol**: TCP
     - **Port**: 5439
     - **Source**: My IP (recommended for security).

       (If sharing with a team, allow a range of IPs or 0.0.0.0/0 for testing only — less secure).
4. Save rules.

---

4. Get the Cluster Endpoint

- Go back to the cluster details.
- Copy the **Endpoint** (e.g., my-redshift-cluster.abc123xyz.us-east-1.redshift.amazonaws.com:5439).

  (Don't include the :5439 when entering into some SQL clients; just the hostname.)

---

5. Connect to Redshift

You can connect using:

Option A: psql (PostgreSQL client)

```
psql -h my-redshift-cluster.abc123xyz.us-east-1.redshift.amazonaws.com \
  -U adminuser \
  -d dev \
  -p 5439
```

It will ask for your password.

Option B: SQL Workbench/J

1. Download **SQL Workbench/J**.
2. Add Redshift JDBC driver.
3. Connection details:
   - Driver: **Amazon Redshift (JDBC)**
   - URL: jdbc:redshift://<endpoint>:5439/dev
   - Username: adminuser
   - Password: (your password)

At this stage, you should be able to run queries like:

```
SELECT current_database(), current_user;
```

1. Install PostgreSQL Client (psql)

Option A: On Local Machine

- **Mac (Homebrew):**
- brew install postgresql
- **Ubuntu/Debian:**
- sudo apt update
- sudo apt install postgresql-client -y
- **Windows:**
  - Download PostgreSQL installer from EnterpriseDB.
  - During setup, select **Command Line Tools** only (no need for full server).
  - After install, psql.exe will be in C:\Program Files\PostgreSQL\<version>\bin.

Option B: On an EC2 Instance

1. Launch an EC2 (Amazon Linux 2 or Ubuntu).
2. Install psql:
   - **Amazon Linux 2:**
   - sudo yum install postgresql -y
   - **Ubuntu:**
   - sudo apt update
   - sudo apt install postgresql-client -y

---

2. Connect to Redshift with psql

You'll need:

- **Cluster endpoint** (from Redshift console → Properties tab).

  Example:
- my-redshift-cluster.abc123xyz.us-east-1.redshift.amazonaws.com
- **Database name**: dev (or the one you created).
- **Master username**: adminuser
- **Port**: 5439

Command:

```
psql -h my-redshift-cluster.abc123xyz.us-east-1.redshift.amazonaws.com \
    -U adminuser \
    -d dev \
    -p 5439
```

It will prompt:

Password for user adminuser:

Enter the password you set during cluster creation.

---

3. Verify Connection

Once inside the psql shell, run:

```
SELECT current_date;
```

Expected output:

```
 current_date
--------------
 2025-09-03
(1 row)
```

This confirms your client is correctly connected to the Redshift cluster.

1. Choose a Public Dataset

You can pick from the AWS Open Data Registry or use something simple for practice.

Example dataset: **NYC Taxi Trips (sample)**

Direct CSV (10MB sample):

https://nyc-tlc.s3.amazonaws.com/trip+data/yellow_tripdata_2023-01.csv

---

2. Download and Prepare Data

On your local machine or an EC2 instance:

```
# create a working folder
mkdir ~/redshift_s3_stage && cd ~/redshift_s3_stage
```

download sample dataset

wget https://nyc-tlc.s3.amazonaws.com/trip+data/yellow_tripdata_2023-01.csv

(if you downloaded a ZIP file, unzip it)

unzip dataset.zip

Now you should have CSV files ready.

---

3. Create Your Private S3 Bucket

1. Go to **AWS Console → S3 → Create bucket**.
    - Name: my-redshift-staging-bucket-<unique-id>
    - Region: same as your Redshift cluster (important for COPY performance).
    - Block **all public access** (keep private).
    - Create bucket.

---

4. Upload Dataset to S3

You can upload via **AWS CLI** or **Console**.

Using AWS CLI:

Make sure AWS CLI is configured (aws configure).

aws s3 cp yellow_tripdata_2023-01.csv s3://my-redshift-staging-bucket-<unique-id>/datasets/

Now the file is in:

s3://my-redshift-staging-bucket-<unique-id>/datasets/yellow_tripdata_2023-01.csv

---

5. Verify Upload

List objects in the bucket:

aws s3 ls s3://my-redshift-staging-bucket-<unique-id>/datasets/

Expected output:

2025-09-03 12:34:56 50000000 yellow_tripdata_2023-01.csv

---

6. Understanding the Role of S3

- **S3 = staging layer**: Instead of loading data row-by-row, Redshift uses **parallel COPY commands** to ingest large datasets efficiently from S3.
- Benefit: Data is loaded in bulk, distributed across nodes, and optimized for analytics.
- Flow: **Raw Dataset → S3 Bucket → COPY → Redshift Tables**.

At this point, you have:

- A **private S3 bucket**.
- A **dataset staged in S3** ready to be loaded into Redshift.

1. Create an IAM Role for Redshift

1. Go to **AWS Console → IAM → Roles → Create role**.
2. **Trusted entity**: Choose **AWS service**.
3. **Use case**: Select **Redshift → Redshift - Customizable**.

   (This ensures the role can be attached to your Redshift cluster.)
4. Click **Next**.

---

2. Attach Policy for S3 Read Access

You have 2 approaches:

Option A: Fine-grained access (best practice)

- Choose **Create policy** → JSON editor:
- {
- "Version": "2012-10-17",
- "Statement": [
- {
- "Effect": "Allow",
- "Action": [
- "s3:GetObject",
- "s3:ListBucket"
-],
- "Resource": [
- "arn:aws:s3:::my-redshift-staging-bucket-<unique-id>",
- "arn:aws:s3:::my-redshift-staging-bucket-<unique-id>/*"
-]
- }
-]
- }
- Save as RedshiftS3ReadOnlyPolicy.
- Attach this policy to your new role.

Option B: Quick & broad access (for learning)

Attach the **AmazonS3ReadOnlyAccess** managed policy.

(Not as secure because it allows read to *all* S3 buckets.)

---

3. Name & Create Role

- Role name: RedshiftS3ReadRole
- Create role.

---

4. Attach IAM Role to Redshift Cluster

1. Go to **Amazon Redshift → Clusters → your-cluster**.
2. Under **Properties → Permissions**, click **Manage IAM roles**.
3. Select RedshiftS3ReadRole.
4. Save changes.

Now your Redshift cluster has the IAM Role that allows reading from your S3 bucket.

---

5. Verify Role in Cluster

Connect via psql and run:

```
-- Check IAM role attached
SELECT * FROM pg_catalog.svv_iam_roles;
```

You should see something like:

```
role_name          |  iam_role_arn
--------------------+--------------------------------------------
RedshiftS3ReadRole  | arn:aws:iam::123456789012:role/RedshiftS3ReadRole
```

---

6. Why This Matters

- Without this role, COPY from S3 fails with **"Access Denied"**.
- With it, Redshift can parallel-load data directly from your bucket.
- This is also a lesson in **principle of least privilege**:

  give only s3:GetObject and s3:ListBucket to keep security tight.

Star Schema Design

Fact Table

- **fact_trips** → Stores transactional data (NYC Taxi Trips).

Dimension Tables

1. **dim_date** → Calendar/time attributes.
2. **dim_passenger** → Passenger categories.
3. **dim_payment_type** → Payment methods.
4. **dim_rate_code** → Rate code categories.
5. **dim_vendor** → Vendor/taxi company info.
6. **dim_location** → Pickup & drop-off locations.

DDL Scripts

```
-- ======================
-- DIMENSION TABLES
-- ======================

-- 1. Date Dimension
CREATE TABLE dim_date (
   date_id       INT PRIMARY KEY,            -- surrogate key (YYYYMMDD)
   full_date     DATE NOT NULL,
   day           SMALLINT NOT NULL,
   month         SMALLINT NOT NULL,
   year          INT NOT NULL,
   day_of_week   VARCHAR(10),
   is_weekend    BOOLEAN
);
```

```sql
-- 2. Passenger Dimension
CREATE TABLE dim_passenger (
    passenger_id   INT PRIMARY KEY,             -- surrogate key
    passenger_count INT NOT NULL,
    description    VARCHAR(50)                  -- e.g., 'Solo', 'Group'
);

-- 3. Payment Type Dimension
CREATE TABLE dim_payment_type (
    payment_type_id  INT PRIMARY KEY,
    payment_type     VARCHAR(20) NOT NULL,       -- e.g., 'Cash', 'Card'
    description      VARCHAR(100)
);

-- 4. Rate Code Dimension
CREATE TABLE dim_rate_code (
    rate_code_id     INT PRIMARY KEY,
    rate_code_desc   VARCHAR(50)
);

-- 5. Vendor Dimension
CREATE TABLE dim_vendor (
    vendor_id        INT PRIMARY KEY,
    vendor_name      VARCHAR(50)
);

-- 6. Location Dimension
CREATE TABLE dim_location (
    location_id      INT PRIMARY KEY,
    borough          VARCHAR(50),
    zone             VARCHAR(100),
    service_zone     VARCHAR(50)
```

```
);

-- ======================
-- FACT TABLE
-- ======================

-- 7. Trips Fact Table
CREATE TABLE fact_trips (
    trip_id          BIGINT IDENTITY(1,1) PRIMARY KEY,   -- surrogate key
    vendor_id        INT REFERENCES dim_vendor(vendor_id),
    pickup_datetime  TIMESTAMP NOT NULL,
    dropoff_datetime TIMESTAMP NOT NULL,
    date_id          INT REFERENCES dim_date(date_id),   -- links to calendar
    passenger_id     INT REFERENCES dim_passenger(passenger_id),
    rate_code_id     INT REFERENCES dim_rate_code(rate_code_id),
    pickup_location_id INT REFERENCES dim_location(location_id),
    dropoff_location_id INT REFERENCES dim_location(location_id),
    payment_type_id  INT REFERENCES dim_payment_type(payment_type_id),
    trip_distance    DECIMAL(8,2),
    fare_amount      DECIMAL(10,2),
    extra            DECIMAL(10,2),
    mta_tax          DECIMAL(10,2),
    tip_amount       DECIMAL(10,2),
    tolls_amount     DECIMAL(10,2),
    total_amount     DECIMAL(10,2)
);
```

Notes on Design

- **Surrogate keys (INT/IDENTITY)** → used for dimensions (faster joins).
- **Fact table uses foreign keys** to link to dimensions.
- **TIMESTAMP** chosen for pickup/dropoff times.
- **DECIMAL(10,2)** for monetary values to avoid rounding errors.
- **Boolean** for quick filters (e.g., weekend flag).
- **Star schema = denormalized for analytics**, unlike OLTP.

With this DDL, your schema is **physically implemented in Redshift**.

Next, you can:

1. Load **dimension tables** first (from lookup datasets).
2. Then load the **fact_trips** table using COPY from your S3 bucket.

General COPY Command Structure

```
COPY table_name
FROM 's3://bucket-name/path/to/file.csv'
IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftS3ReadRole'
DELIMITER ','
IGNOREHEADER 1
REGION 'us-east-1'
TIMEFORMAT 'auto'
;
```

COPY Commands for Each Table

1. dim_date

```
COPY dim_date
FROM 's3://my-redshift-staging-bucket/datasets/dim_date.csv'
IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftS3ReadRole'
DELIMITER ','
IGNOREHEADER 1
REGION 'us-east-1'
TIMEFORMAT 'auto';
```

---

2. dim_passenger

```
COPY dim_passenger
FROM 's3://my-redshift-staging-bucket/datasets/dim_passenger.csv'
IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftS3ReadRole'
DELIMITER ','
IGNOREHEADER 1
REGION 'us-east-1'
TIMEFORMAT 'auto';
```

---

3. dim_payment_type

```
COPY dim_payment_type
FROM 's3://my-redshift-staging-bucket/datasets/dim_payment_type.csv'
IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftS3ReadRole'
DELIMITER ','
IGNOREHEADER 1
REGION 'us-east-1'
TIMEFORMAT 'auto';
```

---

4. dim_rate_code

```
COPY dim_rate_code
FROM 's3://my-redshift-staging-bucket/datasets/dim_rate_code.csv'
IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftS3ReadRole'
DELIMITER ','
IGNOREHEADER 1
REGION 'us-east-1'
TIMEFORMAT 'auto';
```

---

5. dim_vendor

```
COPY dim_vendor
FROM 's3://my-redshift-staging-bucket/datasets/dim_vendor.csv'
IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftS3ReadRole'
```

```
DELIMITER ','
IGNOREHEADER 1
REGION 'us-east-1'
TIMEFORMAT 'auto';
```

---

6. dim_location

```
COPY dim_location
FROM 's3://my-redshift-staging-bucket/datasets/dim_location.csv'
IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftS3ReadRole'
DELIMITER ','
IGNOREHEADER 1
REGION 'us-east-1'
TIMEFORMAT 'auto';
```

---

7. fact_trips

```
COPY fact_trips
FROM 's3://my-redshift-staging-bucket/datasets/fact_trips.csv'
IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftS3ReadRole'
DELIMITER ','
IGNOREHEADER 1
REGION 'us-east-1'
TIMEFORMAT 'auto';
```

Best Practices

- Always **load dimensions first**, then **facts** (to avoid FK constraint issues).
- Use **GZIP compression** in S3 for faster COPY:
- COPY fact_trips
- FROM 's3://my-redshift-staging-bucket/datasets/fact_trips.csv.gz'
- IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftS3ReadRole'
- DELIMITER ','
- IGNOREHEADER 1
- REGION 'us-east-1'
- TIMEFORMAT 'auto'
- GZIP;
- Check row counts after load:

SELECT COUNT(*) FROM fact_trips;

1. Open Query Editor v2

1. Go to the **AWS Management Console → Redshift → Query Editor v2**.
2. Choose your **Redshift cluster**.
3. Enter your **database credentials** (DB name, username, password).
    - This opens a SQL editor pane in the browser.

---

2. Choose a Table to Load

Let's load **dim_vendor** (simple lookup table) for this demo.

Assume you already have a CSV in S3, e.g.:

s3://my-redshift-staging-bucket/datasets/dim_vendor.csv

---

3. Run the COPY Command in Query Editor v2

Paste this into the editor and run:

```
COPY dim_vendor
FROM 's3://my-redshift-staging-bucket/datasets/dim_vendor.csv'
IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftS3ReadRole'
DELIMITER ','
IGNOREHEADER 1
REGION 'us-east-1'
TIMEFORMAT 'auto';
```

---

4. Validate the Load

Run a quick query to check rows:

```
SELECT * FROM dim_vendor LIMIT 10;
```

Expected: You'll see your vendor table populated with rows like:

```
vendor_id | vendor_name
----------+------------
1         | Creative Mobile Technologies
2         | VeriFone Inc.
```

---

5. Why Use Query Editor v2?

- No need to install **psql** or configure local tools.
- Great for **ad-hoc queries**, quick COPY tests, or debugging.
- Can save SQL scripts and share them with your team.
- Best for **learning, prototyping, and validation**.
- For **production-scale loads**, CLI or automated pipelines are more efficient.

1. Row Count Validation

For each of the seven tables, run:

```
-- Dimension tables
SELECT COUNT(*) FROM dim_date;
SELECT COUNT(*) FROM dim_passenger;
SELECT COUNT(*) FROM dim_payment_type;
SELECT COUNT(*) FROM dim_rate_code;
SELECT COUNT(*) FROM dim_vendor;
SELECT COUNT(*) FROM dim_location;

-- Fact table
SELECT COUNT(*) FROM fact_trips;
```

Expected results:

- Dimension tables: smaller row counts (tens to hundreds).
- Fact table: large row count (hundreds of thousands or millions, depending on dataset).

---

2. Data Quality Check (Sample Rows)

Run **LIMIT queries** to eyeball the data:

```
-- Date Dimension
SELECT * FROM dim_date LIMIT 5;

-- Passenger Dimension
SELECT * FROM dim_passenger LIMIT 5;

-- Payment Type Dimension
SELECT * FROM dim_payment_type LIMIT 5;

-- Rate Code Dimension
SELECT * FROM dim_rate_code LIMIT 5;

-- Vendor Dimension
SELECT * FROM dim_vendor LIMIT 5;

-- Location Dimension
SELECT * FROM dim_location LIMIT 5;

-- Fact Table
SELECT * FROM fact_trips LIMIT 5;
```

---

3. What to Look For

- **dim_date** → Check that date_id = YYYYMMDD, full dates are correct, and weekends are flagged properly.
- **dim_passenger** → Passenger categories are consistent (1 = Solo, 2 = Couple, etc.).
- **dim_payment_type** → Cash/Card/Other types are populated.
- **dim_rate_code** → Codes map correctly to descriptions.
- **dim_vendor** → Vendor names aren't truncated.
- **dim_location** → Boroughs/Zones make sense.
- **fact_trips** → Pickup/dropoff timestamps load properly, decimals are not truncated, and foreign keys (e.g., vendor_id) exist in their dimension tables.

---

4. (Optional) Cross-Validation Queries

Check consistency between fact and dimension tables:

```
-- Ensure all vendor_ids in fact table exist in dim_vendor
SELECT DISTINCT vendor_id
FROM fact_trips
WHERE vendor_id NOT IN (SELECT vendor_id FROM dim_vendor);

-- Check passenger_id mapping
SELECT DISTINCT passenger_id
FROM fact_trips
WHERE passenger_id NOT IN (SELECT passenger_id FROM dim_passenger);
```